
رصد لحضور وغياب الطلاب

Rasd: Student Attendance System

الكورس: البرمجة ومدخل لبنية البيانات

إعداد الطالب: أحمد الزييلي عبد الرحمن

المقدمة - Introduction

What is Rasd? | ما هو نظام رصد؟

نظام بسيط وفعال لإدارة وتسجيل حضور وغياب الطلاب في بيئة
(Console Application) نصية.

*A simple and efficient system for managing and recording student attendance in a
console application environment.*

Project Objectives - أهداف المشروع

توفير حلول تقنية بسيطة وفعالة لإدارة الحضور الدراسي.

Providing simple and effective technical solutions for managing academic attendance.



Focus on simplicity and coding efficiency / تركيز على البساطة والكفاءة البرمجية

- **سهولة الاستخدام**
توفير أداة بسيطة ومباشرة لتسجيل حضور الطلاب دون تعقيد.
- **Easy to Use**
Providing a simple and direct tool for recording student attendance without complexity.
- **تقارير واضحة**
عرض حالة الحضور والغياب بشكل موجز ودقيق.
- **Clear Reporting**
Displaying attendance status in a concise and accurate manner.

Tools & Technologies - الأدوات والتقنيات

"البرمجة ليست مجرد كتابة كود، بل هي حل للمشكلات بأبسط الطرق"
"Programming is not just writing code, it's solving problems in the simplest ways."

```
32 self.file = None
33 self.fingerprints = set()
34 self.logdupes = True
35 self.debug = debug
36 self.logger = logging.getLogger(__name__)
37 if path:
38     self.file = open(os.path.join(path, "requests.log"),
39                     "a")
40     self.file.seek(0)
41     self.fingerprints.update({s.request() for s in self.requests})
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool("DEBUG_LOGGING")
46     return cls(job_dir(settings), debug)
47
48 def request_seen(self, request):
49     fp = self.request_fingerprint(request)
50     if fp in self.fingerprints:
51         return True
52     self.fingerprints.add(fp)
53     if self.file:
54         self.file.write(fp + os.linesep)
55
56 def request_fingerprint(self, request):
57     return request_fingerprint(request)
```

Powerful and efficient C++ language | القوية والفعالة C++ لغة

- C++ لغة

لغة قوية وفعالة، تم اختيارها لتعلم أساسيات البرمجة وهياكل البيانات.

- C++ Language

A powerful and efficient language, chosen for learning programming fundamentals and data structures.

- تطبيق الكونسول

واجهة نصية بسيطة تركز على الوظائف الأساسية وسرعة التنفيذ.

- Console Application

A simple text-based interface focusing on core functionalities and execution speed.

Data Structure - هيكل البيانات

كيف يتم تخزين بيانات الطلاب بشكل مرن وفعال.
How student data is stored flexibly and efficiently.

- تمثيل الطالب

لجمع بيانات الطالب (الاسم والحالة) في وحدة واحدة **struct** استخدام

- **Student Struct**

Using **struct** to group student data (name and status) into a single unit.

- تخزين مرن

لإدارة قائمة الطلاب بشكل ديناميكي **std::vector** استخدام

- **Flexible Storage**

Using **std::vector** for dynamic student list management.

A `std::string name`  `bool isPresent`

 `std::vector<Student>`

```
struct Student {  
    string name;  
    bool isPresent;  
};  
  
vector<Student> students;
```

تمثيل برمجي لهيكل البيانات / Programmatic data representation

System Features - مميزات النظام

وظائف برمجية متكاملة لتبسيط عملية رصد الحضور.
Integrated functions to simplify the attendance process.

- **إضافة الطلاب**
إدخال عدد غير محدود من الطلاب مع تخزين أسمائهم بكل سهولة.
- **Add Students**
Input an unlimited number of students and store their names easily.
- **رصد ذكي**
واجهة تفاعلية لتسجيل حالة الحضور (حاضر/غائب) بلمسة واحدة.
- **Smart Recording**
Interactive interface to record attendance status with one touch.

”البساطة هي قمة الرقي في تصميم الأنظمة البرمجية“ |



Organized and user-friendly interface | واجهة برمجية منظمة وسهلة الاستخدام

⚡
سرعة
Speed

✓
دقة
Accuracy

📱
مرونة
Flexibility

🛡️
أمان
Security

Code Snippet - عرض الكود المصدري

C++ منطق تسجيل الحضور باستخدام لغة
Attendance recording logic using C++.

- منطق الدوران

المرور على كافة الطلاب المسجلين في القائمة بشكل تسلسلي.

- Iteration Logic

Sequentially iterating through all registered students in the list.

- تحديث الحالة

استقبال مدخلات المستخدم وتحديث حالة الحضور فوراً.

↺ For Loop

⌨ User Input

✓ Data Update

≡ Vector Mgmt

|"الكود الجيد هو الكود الذي يمكن للبشر فهمه"

```
for (int i = 0; i < students.size(); i++) {  
    char status;  
    cout << "هل الطالب ["  
        << students[i].name  
        << "] حاضر؟ (y/n): ";  
    cin >> status;  
  
    students[i].isPresent =  
        (status == 'y' || status == 'Y');  
}
```

Snippet of attendance logic / مقتطف من منطق تسجيل الحضور

Conclusion - الخاتمة والرؤية المستقبلية

نهاية المشروع وبداية التطوير المستمر.
Project conclusion and the start of continuous development.

Future vision visualization

- الخلاصة

يمثل نظام رصد خطوة أولى ناجحة في بناء أدوات برمجية تحل مشاكل حقيقية ببساطة.

- Future Vision

Expanding functionality with File I/O, GUI integration, and advanced reporting features.

 File I/O

 GUI Interface

 Edit Features

 Growth

”الرحلة تبدأ بخطوة واحدة، والبرمجة رحلة لا تنتهي” |

Rasd Project: A strong starting point / مشروع رصد: نقطة انطلاق قوية

Thank You - شكراً لكم

لمتابعة عرض مشروع نظام رصد للحضور والغياب

إعداد: أحمد الزييلي عبد الرحمن

"القدرة على التبسيط تعني التخلص من غير الضروري لكي يتحدث الضروري"



ahmad@example.com



github.com/ahmad-alzailai